

Tesis

Mariana Belén Cruz Rodríguez Yofre Hernán García Gómez

Tabla de contenidos

1	Introducción	5
2	Formulación Matemática de una CNN	6
3	Representación Tensorial de Imágenes	7
4	Estructura Funcional de la Red	8
5	Parámetros del Modelo	9
6	Kernels Convolucionales	10
6.1	Definición Formal	10
6.2	Interpretación Geométrica	10
6.3	Compartición de parámetros	11
6.3.1	Reducción del número de parámetros	11
6.3.2	Invarianza traslacional	12
7	Operación de Convolución	13
7.1	Caso Unicanal	13
7.1.1	Interpretación como ventana deslizante	14
7.1.2	Interpretación como producto interno	14
7.1.3	Interpretación geométrica	15
7.1.4	Naturaleza lineal de la operación	15
7.1.5	Relación con la correlación cruzada	15
7.1.6	Dependencia local	16
7.2	Convolución Multicanal	16
7.3	Interpretación como producto interno en espacio tensorial	17
7.4	Interpretación geométrica	18
7.5	Naturaleza lineal	18
7.6	Interpretación estructural	18
7.7	Forma Vectorizada	18
8	Múltiples Filtros	19
8.1	Definición de la salida	19
8.2	Estructura tensorial de la salida	19
8.3	Interpretación funcional	20
8.4	Interpretación como vector de características local	20
8.5	Interpretación geométrica	20
8.6	Ejemplos de filtros	22

8.7	Observación estructural	23
9	Stride y Padding	24
9.1	Stride	24
9.2	Padding	24
9.3	Ejemplo ilustrativo	25
9.4	Función de activación	25
9.4.1	Definición	25
9.4.2	No linealidad	27
9.4.3	Ejemplos de funciones de activación	27
9.4.4	Propiedades fundamentales	28
9.4.5	Formulación funcional	29
9.5	Pooling	29
9.5.1	Max Pooling	29
9.5.2	Average Pooling	30
9.5.3	Propiedades	31
9.5.4	Interpretación	32
9.6	Capas completamente conectadas	32
9.6.1	Vectorización de la entrada	32
9.6.2	Transformación afín	32
9.6.3	Aplicación de la función de activación	33
9.6.4	Interpretación	33
9.6.5	Clasificación	33
9.6.6	Propiedades	33
9.6.7	Observación	33
9.7	Función de pérdida	34
9.7.1	Definición general	34
9.7.2	Clasificación multiclase: Entropía cruzada	34
9.7.3	Interpretación probabilística	35
9.7.4	Caso de regresión: Error cuadrático medio	35
9.7.5	Relación con el entrenamiento	35
9.8	Entrenamiento	35
9.8.1	Método del descenso del gradiente	36
9.8.2	Regla de la cadena y retropropagación	39
9.9	Regularización	40
9.9.1	Sobreajuste	40
9.9.2	Penalización de parámetros (Weight Decay)	40
9.9.3	Dropout	40
9.10	Optimización	43
9.10.1	Descenso por gradiente estocástico (SGD)	43
9.10.2	Mini-batch gradient descent	43
9.10.3	Métodos adaptativos	44
9.10.4	Tasa de aprendizaje	44

10 Caso de estudio	45
10.1 Construcción del conjunto de datos	45
10.1.1 Generación de las muestras	45
10.2 Carga y organización de los datos	45
10.3 Normalización y aumentación de datos	46
11 Arquitectura de la red neuronal convolucional	48
11.1 Primer bloque convolucional	48
11.2 Segundo y tercer bloque convolucional	49
11.3 Clasificador	49
12 Entrenamiento	51
13 Sistema de inferencia interactivo	52
14 Conclusiones	53
15 Referencias	54

1 Introducción

En este trabajo se aborda el problema de **reconocimiento automático de glifos del alfabeto náhuatl** mediante el uso de redes neuronales convolucionales (*Convolutional Neural Networks*, CNN). Este problema puede formularse como una tarea de clasificación de imágenes, en la cual cada glifo es representado mediante una estructura matricial que codifica su información visual.

Los glifos presentan variaciones en forma, trazo y estilo, lo que dificulta su identificación mediante métodos tradicionales. En este contexto, las CNN resultan particularmente adecuadas debido a su capacidad para extraer características espaciales relevantes directamente a partir de los datos, tales como bordes, curvas y patrones geométricos distintivos.

2 Formulación Matemática de una CNN

Una red neuronal convolucional puede modelarse como una aplicación parametrizada:

$$f(\cdot, \theta) : \mathcal{X} \subseteq \mathbb{R}^{H \times W \times C} \longrightarrow \mathbb{R}^k \quad (2.1)$$

donde:

- $\mathcal{X}$ es el espacio de entrada (por ejemplo, imágenes),
- H representa la altura (height) de la imagen, es decir, el número de filas,
- W representa el ancho (width), es decir, el número de columnas,
- C representa el número de canales (channels), que corresponde a la cantidad de valores asociados a cada píxel,
- k es la dimensión del espacio de salida (por ejemplo, número de clases en clasificación),
- θ denota el conjunto total de parámetros entrenables del modelo.

3 Representación Tensorial de Imágenes

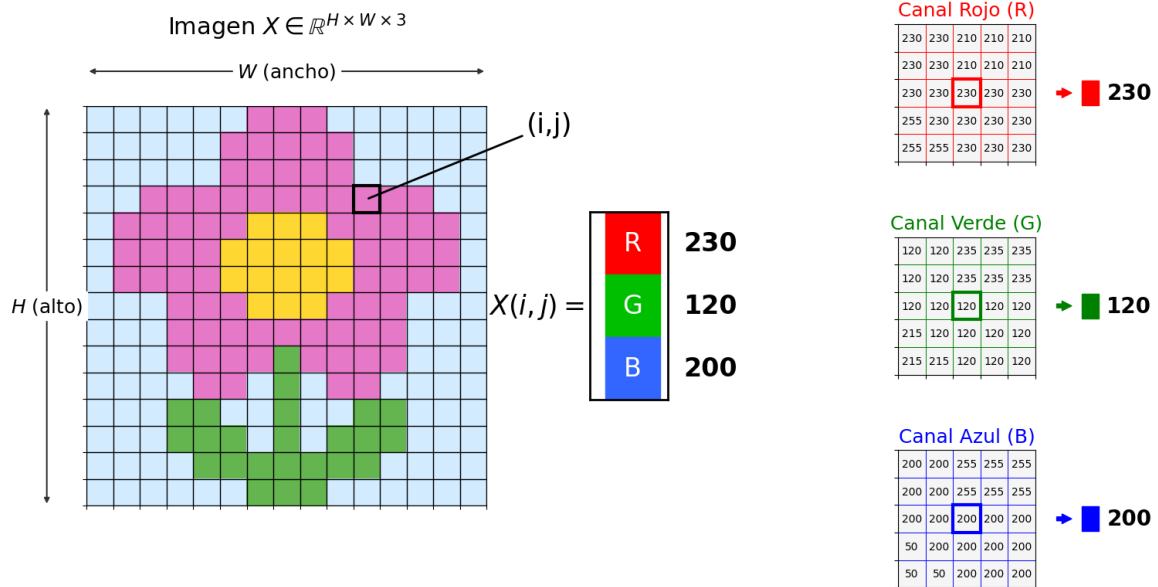
Una imagen digital se representa como un tensor de orden tres:

$$X \in \mathbb{R}^{H \times W \times C} \quad (3.1)$$

Esto implica que para cada posición espacial (i, j) :

$$X(i, j) = (x_1, x_2, \dots, x_C) \in \mathbb{R}^C \quad (3.2)$$

Por tanto, cada píxel contiene un vector que describe información en distintos canales.



4 Estructura Funcional de la Red

Una CNN se construye como la composición de funciones:

$$f(x; \theta) = f^{(L)} \circ f^{(L-1)} \circ \dots \circ f^{(1)}(x) \quad (4.1)$$

donde $L \in \mathbb{N}$ es el número total de capas de la red y cada $f^{(l)}$ representa una transformación asociada a la capa l -ésima.

Asimismo, los parámetros globales se expresan como:

$$\theta = \{\theta^{(1)}, \theta^{(2)}, \dots, \theta^{(L)}\} \quad (4.2)$$

5 Parámetros del Modelo

Los parámetros de una CNN están constituidos principalmente por:

- **Kernels** (filtros convolucionales),
- **Sesgos** asociados a cada filtro.

Estos parámetros determinan completamente la función $f(x; \theta)$ y son ajustados mediante procedimientos de optimización basados en gradiente.

6 Kernels Convolucionales

6.1. Definición Formal

Un kernel es un tensor de parámetros que define un operador lineal local.

Caso unicanal:

$$K \in \mathbb{R}^{k_h \times k_w} \quad (6.1)$$

cuyos elementos se indexan como:

$$K(u, v), \quad u \in \{0, \dots, k_h - 1\}, \quad v \in \{0, \dots, k_w - 1\}$$

Caso multicanal:

$$K \in \mathbb{R}^{k_h \times k_w \times C} \quad (6.2)$$

con entradas:

$$K(u, v, c), \quad u \in \{0, \dots, k_h - 1\}, \quad v \in \{0, \dots, k_w - 1\}, \quad c \in \{1, \dots, C\}$$

6.2. Interpretación Geométrica

Sea $X_{(i,j)}$ una subregión de la imagen obtenida al centrar el kernel en la posición (i, j) , cuyos elementos están dados por:

$$X(i + u, j + v, c)$$

para los mismos índices (u, v, c) definidos anteriormente.

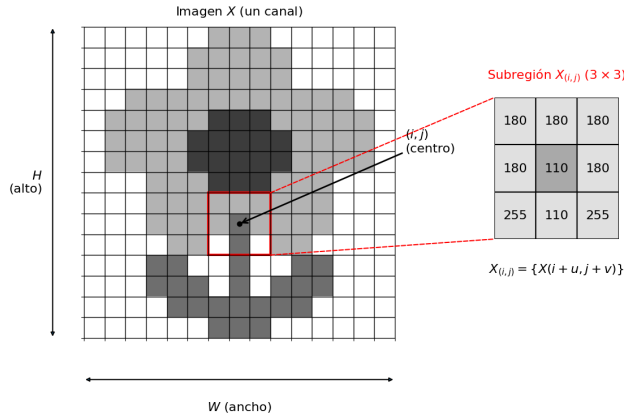
El producto interno:

$$\langle K, X_{(i,j)} \rangle \quad (6.3)$$

mide la similitud entre el patrón definido por el kernel y la región local de la imagen.

1. Subregión de la imagen $X_{(i,j)}$

Sea $X_{(i,j)} = \{X(i + u, j + v, c)\}$



2. Kernel (patrón)

K (3×3)

-1	0	1
-1	0	1
-1	0	1

3. Producto interno

$\langle K, X_{(i,j)} \rangle$

= suma de productos elemento a elemento

$(-1) \cdot 180 = -180$	$(0) \cdot 180 = 0$	$(1) \cdot 180 = 180$
$(-1) \cdot 180 = -180$	$(0) \cdot 110 = 0$	$(1) \cdot 180 = 180$
$(-1) \cdot 255 = -255$	$(0) \cdot 110 = 0$	$(1) \cdot 255 = 255$

Suma total = 0

6.3. Compartición de parámetros

Una propiedad fundamental de las redes neuronales convolucionales es la **compartición de parámetros**, la cual establece que los coeficientes del kernel no dependen de la posición espacial en la que se aplica la operación.

Formalmente, si K es un kernel convolucional, entonces:

$K(u, v, c)$ es independiente de la posición (i, j)

Esto implica que el mismo kernel es utilizado para procesar todas las regiones locales de la entrada.

6.3.1. Reducción del número de parámetros

En ausencia de compartición de parámetros, cada posición (i, j) requeriría un conjunto distinto de pesos, lo cual conduciría a un número total de parámetros del orden:

$$H \cdot W \cdot k_h \cdot k_w \cdot C$$

Sin embargo, al emplear un único kernel compartido, el número de parámetros se reduce a:

$$k_h \cdot k_w \cdot C$$

lo cual representa una reducción significativa en la complejidad del modelo.

6.3.2. Invarianza traslacional

La compartición de parámetros induce una propiedad de **equivarianza por traslación**. Sea $\tau_{a,b}$ un operador de traslación definido sobre la entrada. Entonces, la operación de convolución satisface:

$$\mathcal{T}_K(\tau_{a,b}X) = \tau_{a,b}(\mathcal{T}_KX) \quad (6.4)$$

Esto implica que si una característica aparece en diferentes posiciones espaciales, el modelo será capaz de detectarla de manera consistente.

En consecuencia, la red no depende de la posición absoluta de los patrones, sino únicamente de su estructura local.

7 Operación de Convolución

La operación fundamental en una red neuronal convolucional es la denominada **convolución discreta**. No obstante, es importante señalar que en el contexto de aprendizaje profundo, la operación implementada corresponde en realidad a la **correlación cruzada**, ya que no se realiza la inversión del kernel. A pesar de esta diferencia técnica, en la literatura se mantiene el término “convolución” por convención.

Esta operación define un mecanismo mediante el cual se evalúa la similitud entre un patrón local (kernel) y distintas regiones de la imagen de entrada.

7.1. Caso Unicanal

Sea una imagen unicanal representada por:

$$X \in \mathbb{R}^{H \times W}$$

y un kernel:

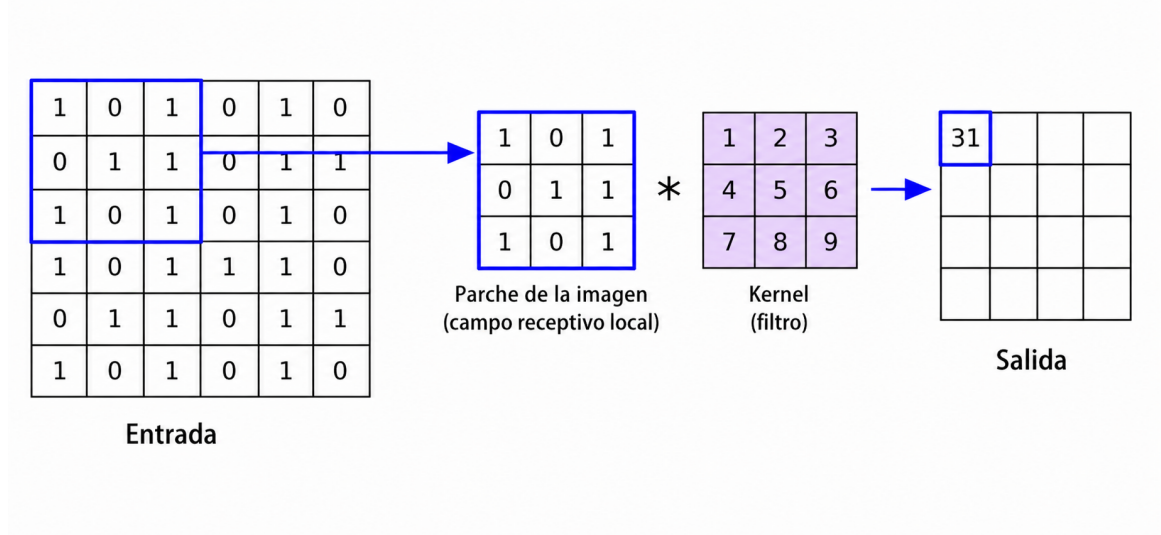
$$K \in \mathbb{R}^{k_h \times k_w}$$

La operación de convolución produce una salida Y , definida en cada posición (i, j) como:

$$Y(i, j) = \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} K(u, v) X(i+u, j+v) \quad (7.1)$$

donde:

- (i, j) denota la posición de la ventana deslizante,
- (u, v) recorre las coordenadas internas del kernel,
- $X(i+u, j+v)$ representa la región local de la imagen.



7.1.1. Interpretación como ventana deslizante

La ecuación anterior describe un proceso en el cual el kernel se desplaza sistemáticamente sobre la imagen. En cada posición, se selecciona una submatriz de X de dimensiones $k_h \times k_w$, sobre la cual se realiza una operación de agregación ponderada.

Formalmente, definimos la submatriz local como:

$$X_{(i,j)} = \{X(i+u, j+v)\}_{u,v}$$

Por lo tanto, la salida en (i, j) depende únicamente de una vecindad local de la imagen, lo cual establece la propiedad de **conectividad local**.

7.1.2. Interpretación como producto interno

La expresión de la convolución puede reescribirse como un producto interno en el espacio vectorial $\mathbb{R}^{k_h \times k_w}$:

$$Y(i, j) = \langle K, X_{(i,j)} \rangle \quad (7.2)$$

donde el producto interno se define como:

$$\langle A, B \rangle = \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} A(u, v) B(u, v)$$

Esta formulación pone de manifiesto que la convolución mide el grado de alineación entre el kernel y la región local.

7.1.3. Interpretación geométrica

Desde un punto de vista geométrico, el valor $Y(i, j)$ representa la proyección del vector $X_{(i,j)}$ sobre el vector K en el espacio euclidiano de dimensión $k_h k_w$. En consecuencia:

- Si $X_{(i,j)}$ es similar a K , entonces $Y(i, j)$ será grande.
- Si son ortogonales, el resultado será cercano a cero.
- Si son opuestos, el valor será negativo.

Esto permite interpretar al kernel como un **detector de patrones**.

7.1.4. Naturaleza lineal de la operación

La convolución es un operador lineal. Es decir, para cualesquiera tensores X_1, X_2 y escalares α, β , se cumple:

$$\mathcal{T}_K(\alpha X_1 + \beta X_2) = \alpha \mathcal{T}_K(X_1) + \beta \mathcal{T}_K(X_2) \quad (7.3)$$

donde $\mathcal{T}_K$ denota el operador de convolución asociado al kernel K .

7.1.5. Relación con la correlación cruzada

Es importante destacar que la definición utilizada en redes neuronales corresponde a la correlación cruzada:

$$Y(i, j) = \sum_{u,v} K(u, v) X(i + u, j + v)$$

mientras que la convolución clásica implicaría una inversión del kernel:

$$Y(i, j) = \sum_{u,v} K(u, v) X(i - u, j - v)$$

Sin embargo, dado que los parámetros del kernel son aprendidos durante el entrenamiento, ambas formulaciones resultan equivalentes en términos de capacidad representacional.

7.1.6. Dependencia local

Finalmente, se observa que:

$$Y(i, j) \text{ depende únicamente de } X_{(i, j)}$$

lo cual implica que cada salida está influenciada únicamente por una región local de la entrada. Esta propiedad es clave para la eficiencia computacional y la capacidad de generalización del modelo.

7.2. Convolución Multicanal

En el caso general, las imágenes no son unicanal, sino que poseen múltiples canales (por ejemplo, RGB). En consecuencia, la operación de convolución debe extenderse para considerar simultáneamente la información presente en cada canal.

Sea:

$$X \in \mathbb{R}^{H \times W \times C}$$

una imagen con C canales, y sea un kernel:

$$K \in \mathbb{R}^{k_h \times k_w \times C}$$

donde cada canal del kernel corresponde a un canal de la entrada.

La salida de la convolución en la posición (i, j) está dada por:

$$Y(i, j) = \sum_{c=1}^C \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} K(u, v, c) X(i+u, j+v, c) \quad (7.4)$$

Esta expresión indica que la contribución de cada canal se calcula de manera independiente y posteriormente se agregan (suman) todas las contribuciones.

La operación anterior puede interpretarse como la suma de convoluciones unicanal:

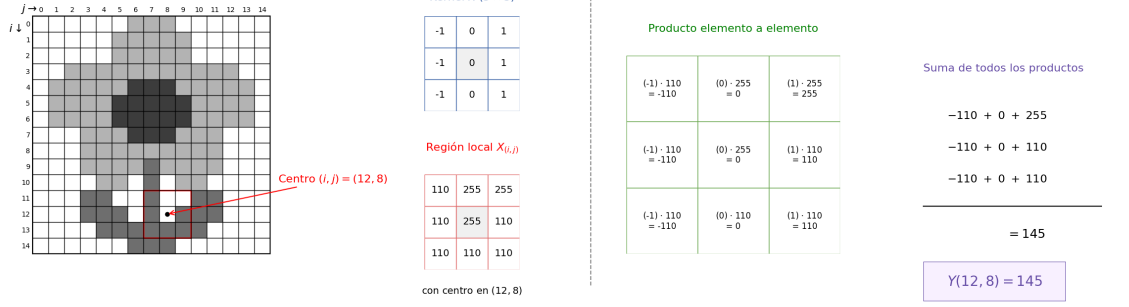
$$Y(i, j) = \sum_{c=1}^C Y^{(c)}(i, j)$$

donde:

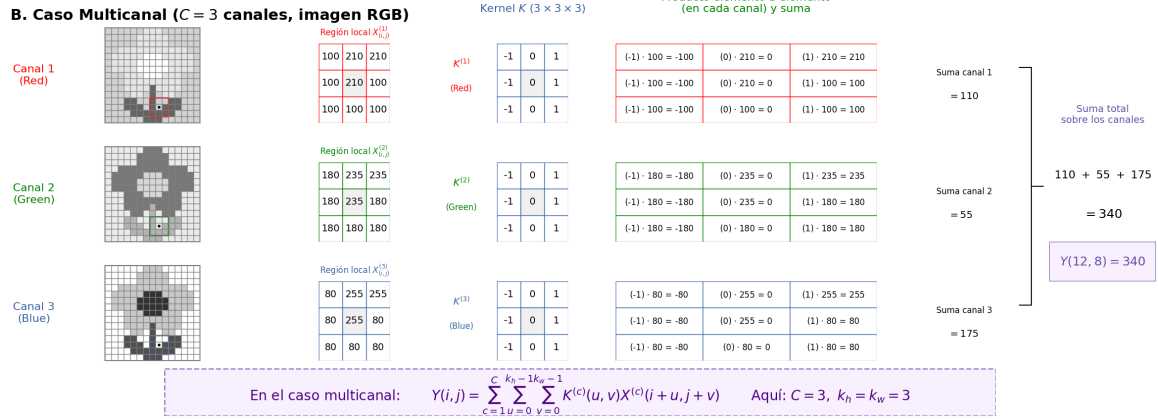
$$Y^{(c)}(i, j) = \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} K(u, v, c) X(i+u, j+v, c)$$

Cada término $Y^{(c)}(i, j)$ representa la contribución del canal c al valor final de la salida.

A. Caso Unicanal (1 canal en Escala de Grises)



B. Caso Multicanal ($C = 3$ canales, imagen RGB)



7.3. Interpretación como producto interno en espacio tensorial

Sea $X_{(i,j)}$ la subregión local de la imagen centrada en (i, j) . Entonces:

$$X_{(i,j)} \in \mathbb{R}^{k_h \times k_w \times C}$$

Al vectorizar ambos tensores:

$$\text{vec}(X_{(i,j)}) \in \mathbb{R}^{k_h k_w C}, \quad \text{vec}(K) \in \mathbb{R}^{k_h k_w C}$$

la convolución puede escribirse como:

$$Y(i, j) = \text{vec}(K)^\top \text{vec}(X_{(i,j)}) \quad (7.5)$$

Esto muestra que la operación es un producto interno en un espacio euclidiano de dimensión $k_h k_w C$.

7.4. Interpretación geométrica

Desde un punto de vista geométrico, cada región local de la imagen se interpreta como un punto en el espacio $\mathbb{R}^{k_h k_w C}$, y el kernel define una dirección en dicho espacio.

Por tanto:

- valores grandes de $Y(i, j)$ indican alta alineación entre el patrón y la región,
- valores cercanos a cero indican baja correlación,
- valores negativos indican oposición estructural.

En consecuencia, el kernel actúa como un detector de patrones multicanal.

7.5. Naturaleza lineal

La convolución multicanal es un operador lineal. Es decir, para cualesquiera tensores X_1, X_2 y escalares α, β , se cumple:

$$\mathcal{T}_K(\alpha X_1 + \beta X_2) = \alpha \mathcal{T}_K(X_1) + \beta \mathcal{T}_K(X_2)$$

donde $\mathcal{T}_K$ denota el operador de convolución asociado al kernel K .

7.6. Interpretación estructural

Es importante destacar que, a diferencia del caso unicanal, el kernel multicanal no detecta únicamente patrones espaciales, sino también **correlaciones entre canales**.

Por ejemplo, en imágenes RGB, el kernel puede aprender a identificar combinaciones específicas de intensidades en los canales rojo, verde y azul, lo cual permite detectar patrones más complejos que aquellos basados únicamente en estructura espacial.

7.7. Forma Vectorizada

$$Y(i, j) = K^\top X_{(i, j)}$$

8 Múltiples Filtros

En una red neuronal convolucional, no se utiliza un único kernel, sino un conjunto de filtros que permiten extraer distintas características de la entrada. Este conjunto se denota como:

$$\{K^{(f)}\}_{f=1}^F, \quad K^{(f)} \in \mathbb{R}^{k_h \times k_w \times C} \quad (8.1)$$

donde F representa el número total de filtros.

8.1. Definición de la salida

Cada filtro $K^{(f)}$ actúa de manera independiente sobre la entrada y produce un mapa de activación asociado. Para cada posición (i, j) , la salida correspondiente al filtro f está dada por:

$$Y_f(i, j) = \langle K^{(f)}, X_{(i, j)} \rangle \quad (8.2)$$

Por lo tanto, a cada filtro le corresponde una función:

$$Y_f : \{1, \dots, H'\} \times \{1, \dots, W'\} \rightarrow \mathbb{R}$$

8.2. Estructura tensorial de la salida

Al considerar todos los filtros simultáneamente, la salida completa se organiza como un tensor de orden tres:

$$Y \in \mathbb{R}^{H' \times W' \times F} \quad (8.3)$$

donde:

- H' y W' son las dimensiones espaciales de la salida,
- F es el número de canales de salida, también llamados **mapas de características**.

En particular, la componente $Y(:, :, f)$ corresponde al mapa de activación generado por el filtro $K^{(f)}$.

8.3. Interpretación funcional

Cada filtro puede interpretarse como un detector especializado en cierto tipo de patrón. En consecuencia, la colección de filtros define una familia de funciones:

$$\{\mathcal{T}_{K^{(f)}}\}_{f=1}^F$$

donde cada operador $\mathcal{T}_{K^{(f)}}$ extrae una característica distinta de la entrada.

De este modo, la salida de la capa puede interpretarse como la aplicación conjunta de múltiples detectores, produciendo una representación enriquecida de la imagen.

8.4. Interpretación como vector de características local

Para cada posición (i, j) , la salida puede agruparse como un vector:

$$Y(i, j) = (Y_1(i, j), Y_2(i, j), \dots, Y_F(i, j)) \in \mathbb{R}^F \quad (8.4)$$

Esto implica que cada posición espacial de la imagen original es transformada en un vector de características de dimensión F .

Por tanto, la capa convolucional realiza una transformación:

$$\mathbb{R}^{k_h \times k_w \times C} \longrightarrow \mathbb{R}^F$$

aplicada localmente en cada vecindad de la imagen.

8.5. Interpretación geométrica

Desde una perspectiva geométrica, cada filtro $K^{(f)}$ define una dirección en el espacio $\mathbb{R}^{k_h k_w C}$. En consecuencia, el vector $Y(i, j)$ contiene las proyecciones de la región local $X_{(i, j)}$ sobre cada una de estas direcciones.

Esto permite interpretar la salida como una representación de la información local en una base (no necesariamente ortogonal) definida por los filtros aprendidos.

8.6. Ejemplos de filtros

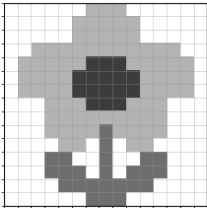
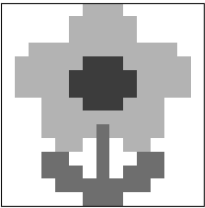
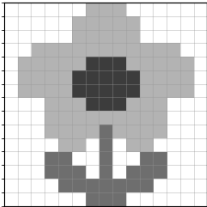
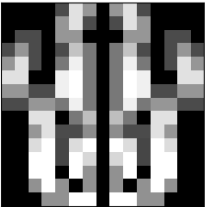
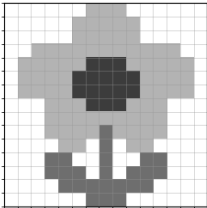
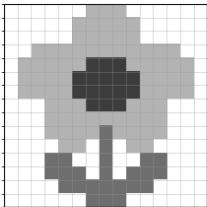
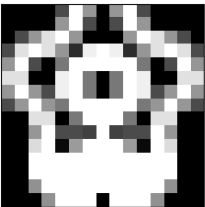
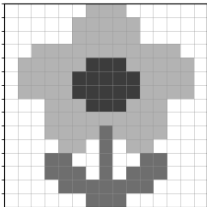
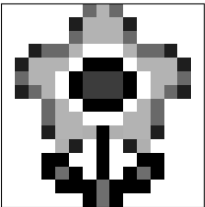
Operación	Filtro (kernel)	Descripción	Imagen Original	Resultado
Identidad (paso)	$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	No altera los píxeles.		
Bordes (horizontal)	$\begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$	Resalta bordes verticales.		
Bordes (vertical)	$\begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$	Resalta bordes horizontales.		
Laplaciano	$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$	Resalta cambios de intensidad.		
Enfoque	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	Acentúa detalles y bordes.		

Figura 8.1: Los distintos filtros convolucionales permiten detectar características específicas en la imagen de entrada. En la imagen se muestran ejemplos representativos del efecto de diferentes kernels sobre una misma imagen.

8.7. Observación estructural

Es importante notar que el número de filtros F determina la capacidad de representación de la capa. Un mayor número de filtros permite capturar una mayor variedad de patrones, pero también incrementa el número de parámetros y el costo computacional.

En capas profundas, los filtros tienden a representar características de mayor nivel semántico, construidas a partir de combinaciones de patrones detectados en capas anteriores.

9 Stride y Padding

9.1. Stride

El *stride* s controla el desplazamiento del kernel durante la operación de convolución. Es decir, determina cuántas posiciones se desplaza el kernel en cada paso.

Las dimensiones de salida están dadas por:

$$H' = \left\lfloor \frac{H - k_h + 2p}{s} \right\rfloor + 1, \quad W' = \left\lfloor \frac{W - k_w + 2p}{s} \right\rfloor + 1 \quad (9.1)$$

donde:

- H, W son las dimensiones de la entrada,
- k_h, k_w son las dimensiones del kernel,
- p es el padding,
- s es el stride.

Un valor mayor de s produce una reducción en la resolución espacial de la salida.

9.2. Padding

El *padding* es una operación que consiste en añadir valores (generalmente ceros) alrededor de la entrada antes de aplicar la convolución. Esto se hace para controlar el tamaño de la salida y preservar información en los bordes.

$$X_{\text{pad}} \in \mathbb{R}^{(H+2p) \times (W+2p) \times C}$$

El padding puede interpretarse como una extensión de la función de entrada X fuera de su dominio original. Formalmente, se define la extensión $\tilde{X}$ como:

$$\tilde{X}(i, j) = \begin{cases} X(i, j), & \text{si } (i, j) \text{ pertenece al dominio original,} \\ 0, & \text{en otro caso} \end{cases}$$

Esta extensión corresponde a una extensión por cero del dominio de la función.

En una capa convolucional, el padding se aplica antes de la convolución. Por lo tanto, la operación completa de una capa convolucional con padding se puede expresar como:

$$Y = \sigma((\text{pad}(X)) \star K + b)$$

donde:

- X es el tensor de entrada,
- $\text{pad}(X)$ es la entrada extendida mediante padding,
- $\star$ denota la operación de correlación cruzada,
- K es el kernel o filtro,
- b es el sesgo,
- σ es la función de activación.

9.3. Ejemplo ilustrativo

Para ilustrar el efecto del padding sobre las dimensiones de salida, se presenta el siguiente ejemplo:

9.4. Función de activación

Una función de activación es una aplicación no lineal que se introduce después de una transformación afín, con el objetivo de dotar al modelo de la capacidad de aproximar funciones no lineales.

9.4.1. Definición

Sea $x \in \mathbb{R}^n$ un vector de entrada, $w \in \mathbb{R}^n$ un vector de pesos y $b \in \mathbb{R}$ un sesgo. Se define la preactivación como:

$$z = w^T x + b \tag{9.2}$$

La salida de la neurona está dada por:

$$y = \sigma(z) \tag{9.3}$$

donde $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ es la función de activación.

1. Convolución sin relleno (padding = 0)

Imagen de entrada (4×4)

3	5	2	7
4	1	3	8
6	3	8	2
9	6	1	5

La imagen de entrada tiene tamaño 4×4.

Filtro (3×3)

1	2	1
2	1	2
1	1	2

El filtro (kernel) tiene tamaño 3×3.

Mapa de características de salida (2×2)

55	52
57	50

El mapa de salida tiene tamaño 2×2.

2. Convolución con relleno (padding = 1)

Imagen de entrada con relleno (6×6)

0	0	0	0	0	0
0	3	5	2	7	0
0	4	1	3	8	0
0	6	3	8	2	0
0	9	6	1	5	0
0	0	0	0	0	0

Se agrega un relleno de ceros (padding = 1) alrededor de la imagen original.

Filtro (3×3)

1	2	1
2	1	2
1	1	2

El filtro (kernel) tiene tamaño 3×3.

Mapa de características de salida (4×4)

19	26	46	22
29	55	52	40
42	57	50	43
36	46	44	19

El mapa de salida tiene el mismo tamaño que la imagen original: 4×4.

Figura 9.1: Ejemplo de convolución con y sin padding. La incorporación de padding permite controlar el tamaño de la salida. En particular, cuando se utiliza padding adecuado, es posible preservar las dimensiones espaciales de la imagen original tras la aplicación de la convolución. El color rosa identifica la imagen de entrada y la salida de la convolución, mientras que el verde representa el filtro empleado. En el caso del ejemplo con padding, las celdas correspondientes al relleno se distinguen al estar por fuera del cuadro rojo, para resaltar que no forman parte de la imagen original.

9.4.2. No linealidad

Considérese una red neuronal profunda compuesta únicamente por transformaciones afines. En tal caso, la composición de dichas transformaciones satisface:

$$f(x) = W_L W_{L-1} \cdots W_1 x + b'$$

lo cual sigue siendo una transformación afín. Por consiguiente, sin la introducción de funciones de activación no lineales, la red carece de capacidad expresiva para modelar relaciones no lineales.

En una red neuronal convolucional (CNN), la salida de una capa se expresa como:

$$Y = \sigma(X \star K + b) \tag{9.4}$$

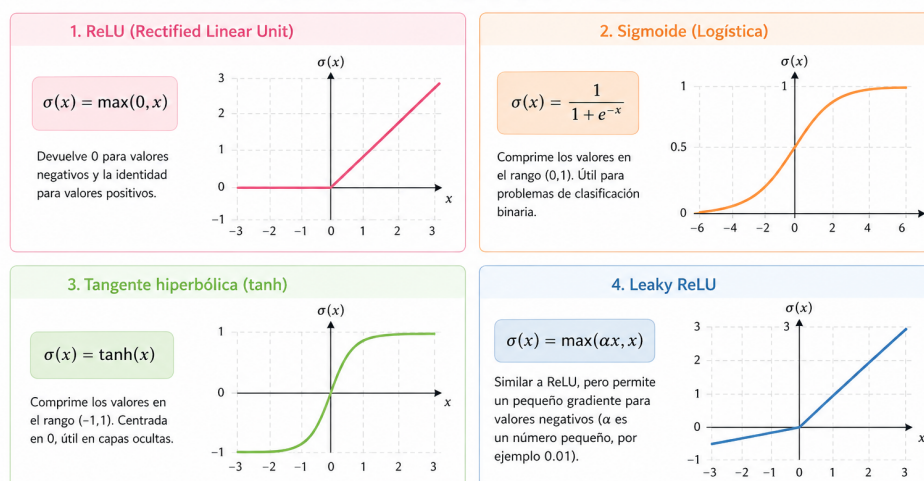
donde:

- X es el tensor de entrada
- $\star$ denota la operación de correlación cruzada
- K es el conjunto de filtros (kernels)
- b es el sesgo
- σ se aplica elemento a elemento

9.4.3. Ejemplos de funciones de activación

Algunas de las funciones de activación más utilizadas son las siguientes:

Funciones de activación comunes



9.4.3.1. ReLU (Rectified Linear Unit)

$$\sigma(x) = \max(0, x) \quad (9.5)$$

9.4.3.2. Sigmoide

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (9.6)$$

9.4.3.3. Tangente hiperbólica

$$\sigma(x) = \tanh(x) \quad (9.7)$$

9.4.4. Propiedades fundamentales

Las funciones de activación empleadas en redes neuronales satisfacen típicamente:

- No linealidad, necesaria para incrementar la capacidad de representación del modelo
- Aplicación puntual (element-wise), preservando la estructura espacial en CNN
- Diferenciabilidad (al menos casi en todas partes), requisito esencial para métodos de optimización basados en gradiente

9.4.5. Formulación funcional

Desde una perspectiva funcional, una red neuronal profunda puede interpretarse como la composición de aplicaciones de la forma:

$$f_{\theta} = \sigma_L \circ T_L \circ \dots \circ \sigma_1 \circ T_1 \quad (9.8)$$

donde cada transformación afín está dada por:

$$T_i(x) = W_i x + b_i$$

9.5. Pooling

El *pooling* es una operación que reduce la dimensión espacial de la representación, conservando las características más relevantes. Esta operación se aplica de manera independiente en cada canal del tensor de entrada.

Sea:

$$X \in \mathbb{R}^{H \times W \times C}$$

la entrada de la capa. El pooling produce una salida:

$$Y \in \mathbb{R}^{H' \times W' \times C}$$

donde las dimensiones H' y W' son menores que H y W .

9.5.1. Max Pooling

El *max pooling* es la forma más común de pooling. Para cada región local $X_{(i,j)}$, se define:

$$Y(i, j, c) = \max_{(u,v) \in \mathcal{R}} X(i + u, j + v, c) \quad (9.9)$$

donde $\mathcal{R}$ representa la ventana de pooling (por ejemplo, de tamaño 2×2).

9.5.2. Average Pooling

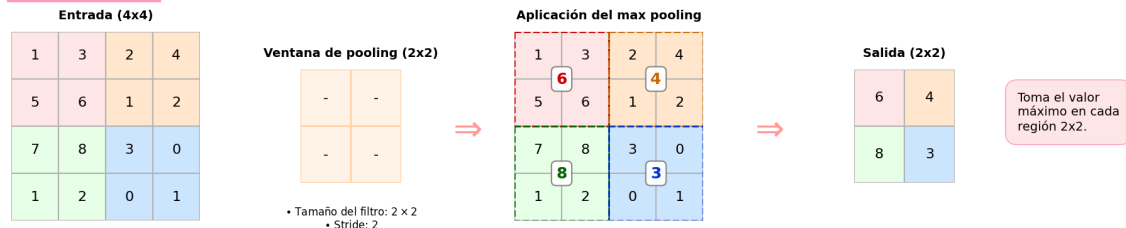
Otra variante es el *average pooling*, definido como:

$$Y(i, j, c) = \frac{1}{|\mathcal{R}|} \sum_{(u, v) \in \mathcal{R}} X(i + u, j + v, c) \quad (9.10)$$

El pooling sirve para reducir las dimensiones espaciales (ancho y alto) de un tensor, asegurando que se conserve la información más relevante. Aplicar esta técnica brinda grandes beneficios a los modelos: disminuye significativamente el costo computacional, introduce invariancia a pequeñas traslaciones en la imagen y ayuda a prevenir el sobreajuste. Además, es importante destacar que esta reducción se aplica de manera independiente a cada canal de profundidad del tensor

Pooling: reducción de dimensión espacial

1. Max Pooling



2. Average Pooling

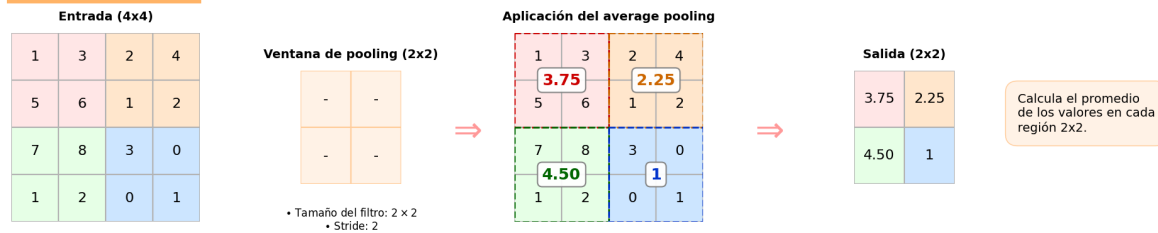


Figura 9.2: En la parte superior de la figura se ilustra el funcionamiento del Max Pooling utilizando una ventana de tamaño 2×2 y un stride de 2. La imagen de entrada se divide en cuatro regiones no superpuestas, identificadas mediante diferentes colores. Para cada región, el operador selecciona únicamente el valor máximo: en la región superior izquierda se obtiene 6, en la superior derecha 4, en la inferior izquierda 8 y en la inferior derecha 3. Estos valores conforman el mapa de salida de tamaño 2×2 , preservando las características más destacadas de cada zona mientras se reduce la dimensionalidad espacial. De manera similar, en la parte inferior de la figura se muestra el proceso de Average Pooling con la misma ventana 2×2 y el mismo stride. En lugar de conservar el valor máximo, se calcula el promedio de los elementos contenidos en cada región. Así, las cuatro ventanas generan los valores 3.75, 2.25, 4.50 y 1, que constituyen la salida reducida. Este método produce una representación más suavizada de la información, manteniendo la tendencia general de los datos presentes en cada región de la imagen.

9.5.3. Propiedades

El pooling presenta las siguientes propiedades:

- Reduce la dimensionalidad espacial, disminuyendo el costo computacional
- Introduce una cierta invariancia a traslaciones pequeñas
- Opera de forma independiente en cada canal

9.5.4. Interpretación

El pooling puede interpretarse como una forma de resumen local de la información. En el caso del max pooling, se selecciona la característica más prominente dentro de cada región, mientras que el average pooling produce una media de las intensidades.

En arquitecturas profundas, el pooling contribuye a la construcción de representaciones más abstractas, al reducir progresivamente la resolución espacial.

9.6. Capas completamente conectadas

Las capas completamente conectadas (*Fully Connected*, FC) constituyen la etapa final de una red neuronal convolucional. Su función es combinar las características extraídas por las capas anteriores para realizar la tarea de clasificación o regresión.

9.6.1. Vectorización de la entrada

Sea:

$$X \in \mathbb{R}^{H \times W \times C}$$

la salida de la última capa convolucional. Esta se transforma en un vector mediante una operación de vectorización:

$$x = \text{vec}(X) \in \mathbb{R}^{HWC} \quad (9.11)$$

9.6.2. Transformación afín

La capa completamente conectada aplica una transformación lineal seguida de un sesgo:

$$z = Wx + b \quad (9.12)$$

donde:

- $W \in \mathbb{R}^{k \times HWC}$ es la matriz de pesos
- $b \in \mathbb{R}^k$ es el vector de sesgos
- k es el número de neuronas en la capa

9.6.3. Aplicación de la función de activación

La salida de la capa está dada por:

$$y = \sigma(z) \quad (9.13)$$

donde σ es una función de activación aplicada componente a componente.

9.6.4. Interpretación

Las capas completamente conectadas pueden interpretarse como clasificadores que operan sobre las características extraídas por la red convolucional. En particular, cada neurona combina toda la información disponible en la representación vectorizada.

Esto contrasta con las capas convolucionales, donde la conectividad es local.

9.6.5. Clasificación

En problemas de clasificación multiclase, la última capa suele utilizar la función *softmax*, definida como:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^k e^{z_j}} \quad (9.14)$$

Esta función transforma el vector z en una distribución de probabilidad sobre las clases.

9.6.6. Propiedades

Las capas completamente conectadas presentan las siguientes características:

- Conectividad global: cada neurona está conectada con todas las entradas
- Alta capacidad de representación
- Mayor número de parámetros en comparación con capas convolucionales

9.6.7. Observación

Debido al elevado número de parámetros, las capas completamente conectadas son más propensas al sobreajuste. Por esta razón, en arquitecturas modernas, su uso se reduce o se reemplaza parcialmente mediante técnicas como *global average pooling*.

9.7. Función de pérdida

La función de pérdida (*loss function*) cuantifica la discrepancia entre las predicciones del modelo y los valores reales. Constituye el criterio fundamental que guía el proceso de entrenamiento, ya que permite evaluar la calidad de las predicciones y definir un objetivo de optimización.

9.7.1. Definición general

Sea $f_\theta(x)$ la salida del modelo para una entrada x , y sea y la etiqueta real asociada. Una función de pérdida se define como:

$$\mathcal{L}(f_\theta(x), y)$$

donde $\mathcal{L} : \mathbb{R}^k \times \mathbb{R}^k \rightarrow \mathbb{R}$ mide el error entre la predicción y el valor objetivo.

Dado un conjunto de datos $\{(x_i, y_i)\}_{i=1}^N$, la función de costo global se define como:

$$J(\theta) = \frac{1}{N} \sum_{i=1}^N \mathcal{L}(f_\theta(x_i), y_i) \quad (9.15)$$

9.7.2. Clasificación multiclase: Entropía cruzada

En problemas de clasificación multiclase, la función de pérdida más utilizada es la entropía cruzada (*cross-entropy*).

Sea $p = f_\theta(x)$ la salida del modelo después de aplicar la función *softmax*, y sea y un vector one-hot que representa la clase verdadera. La pérdida se define como:

$$\mathcal{L}(p, y) = - \sum_{j=1}^k y_j \log(p_j) \quad (9.16)$$

Dado que y es un vector one-hot, esta expresión se reduce a:

$$\mathcal{L}(p, y) = - \log(p_y)$$

donde p_y es la probabilidad asignada a la clase correcta.

9.7.3. Interpretación probabilística

La entropía cruzada puede interpretarse como una medida de divergencia entre la distribución verdadera y y la distribución predicha p . Minimizar esta función equivale a maximizar la probabilidad de observar los datos bajo el modelo.

Desde un punto de vista estadístico, este enfoque corresponde al método de máxima verosimilitud.

9.7.4. Caso de regresión: Error cuadrático medio

En problemas de regresión, una función de pérdida común es el error cuadrático medio (*Mean Squared Error*, MSE), definido como:

$$\mathcal{L}(f_{\theta}(x), y) = \|f_{\theta}(x) - y\|^2 \quad (9.17)$$

Esta función penaliza de manera más severa los errores grandes, lo que favorece ajustes más precisos.

9.7.5. Relación con el entrenamiento

El objetivo del entrenamiento consiste en encontrar los parámetros θ que minimizan la función de costo global:

$$\theta^* = \arg \min_{\theta} J(\theta) \quad (9.18)$$

Este problema se resuelve típicamente mediante métodos de optimización iterativos, como el descenso por gradiente y sus variantes.

9.8. Entrenamiento

El entrenamiento de una red neuronal convolucional consiste en ajustar los parámetros θ del modelo con el objetivo de minimizar la función de pérdida definida previamente.

Dado un conjunto de datos $\{(x_i, y_i)\}_{i=1}^N$, el problema de entrenamiento puede formularse como:

$$\min_{\theta} J(\theta) = \frac{1}{N} \sum_{i=1}^N \mathcal{L}(f_{\theta}(x_i), y_i) \quad (9.19)$$

9.8.1. Método del descenso del gradiente

Considérese el problema de optimización irrestricta

$$\min_{x \in \mathbb{R}^n} f(x),$$

donde $f : \mathbb{R}^n \rightarrow \mathbb{R}$ es una función diferenciable. El objetivo consiste en construir un procedimiento iterativo que permita aproximarse a un punto minimizante de f . Entre los métodos más importantes para resolver este tipo de problemas se encuentra el método del descenso del gradiente, también conocido como *gradient descent* o método del máximo descenso.

La idea fundamental del método proviene de la interpretación geométrica del gradiente. Para una función diferenciable f , el vector gradiente

$$\nabla f(x) = \begin{pmatrix} \frac{\partial f}{\partial x_1}(x) \\ \frac{\partial f}{\partial x_2}(x) \\ \vdots \\ \frac{\partial f}{\partial x_n}(x) \end{pmatrix}$$

indica la dirección de máximo crecimiento local de la función en el punto x . En consecuencia, el vector opuesto

$$-\nabla f(x)$$

corresponde a la dirección de máximo decrecimiento local. El método del descenso del gradiente aprovecha precisamente esta propiedad para generar una sucesión de iterados que produzca una disminución progresiva del valor de la función objetivo.

Partiendo de un punto inicial $x_0 \in \mathbb{R}^n$, el método construye una sucesión $\{x_k\}$ definida mediante la iteración

$$x_{k+1} = x_k - \alpha_k \nabla f(x_k),$$

donde $\alpha_k > 0$ representa el tamaño de paso asociado a la iteración k . La elección de este parámetro resulta fundamental para el desempeño del algoritmo, pues determina la magnitud del desplazamiento realizado en la dirección de descenso.

La justificación matemática del método se obtiene a partir de la noción de dirección de descenso. Sea

$$d_k = -\nabla f(x_k).$$

Entonces,

$$\nabla f(x_k)^T d_k = \nabla f(x_k)^T (-\nabla f(x_k)) = -\|\nabla f(x_k)\|^2.$$

Si $\nabla f(x_k) \neq 0$, se tiene que

$$\|\nabla f(x_k)\|^2 > 0,$$

y por tanto

$$\nabla f(x_k)^T d_k < 0.$$

Esto demuestra que d_k constituye una dirección estricta de descenso para la función f . En otras palabras, existe un valor suficientemente pequeño de $\alpha > 0$ tal que

$$f(x_k + \alpha d_k) < f(x_k).$$

Esta propiedad puede analizarse con mayor detalle mediante el desarrollo de Taylor de primer orden. Para α pequeño se tiene

$$f(x_k + \alpha d_k) = f(x_k) + \alpha \nabla f(x_k)^T d_k + o(\alpha).$$

Sustituyendo $d_k = -\nabla f(x_k)$, resulta

$$f(x_k - \alpha \nabla f(x_k)) = f(x_k) - \alpha \|\nabla f(x_k)\|^2 + o(\alpha).$$

Debido a que el término dominante es negativo, el valor de la función disminuye localmente para valores suficientemente pequeños de α . Esta observación constituye la base teórica del algoritmo.

La selección del tamaño de paso α_k es uno de los aspectos centrales del método. Una posibilidad consiste en utilizar un paso constante,

$$\alpha_k = \alpha,$$

lo cual simplifica considerablemente la implementación. Sin embargo, si el valor de α es demasiado grande, la sucesión puede divergir o presentar oscilaciones; por el contrario, si α es demasiado pequeño, la convergencia puede resultar excesivamente lenta.

Otra estrategia consiste en determinar el tamaño de paso mediante una búsqueda lineal exacta. En este caso, α_k se obtiene resolviendo el problema unidimensional

$$\alpha_k = \arg \min_{\alpha > 0} f(x_k - \alpha \nabla f(x_k)).$$

De esta manera, en cada iteración se minimiza exactamente la función sobre la recta definida por la dirección de descenso. Aunque este enfoque posee una sólida justificación teórica, en problemas de gran dimensión suele resultar costoso desde el punto de vista computacional. Por esta razón, frecuentemente se emplean búsquedas lineales inexactas basadas en criterios como las condiciones de Armijo o de Wolfe, las cuales garantizan una reducción suficiente de la función objetivo sin resolver completamente el subproblema unidimensional.

El criterio de terminación más habitual se basa en la norma del gradiente. Dado que un punto minimizante diferenciable debe satisfacer la condición necesaria de primer orden

$$\nabla f(x^*) = 0,$$

el algoritmo suele detenerse cuando

$$\|\nabla f(x_k)\| < \varepsilon,$$

para cierta tolerancia $\varepsilon > 0$.

Las propiedades de convergencia del método dependen de las características de la función objetivo. Si f es convexa y su gradiente es Lipschitz continuo, entonces la sucesión generada por el método converge hacia un mínimo global. Más aún, si f es fuertemente convexa, es decir, si existe $\mu > 0$ tal que

$$f(y) \geq f(x) + \nabla f(x)^T(y - x) + \frac{\mu}{2}\|y - x\|^2,$$

entonces el método presenta convergencia lineal para elecciones apropiadas del tamaño de paso.

Para ilustrar el procedimiento, considérese la función cuadrática

$$f(x, y) = x^2 + y^2.$$

Su gradiente está dado por

$$\nabla f(x, y) = \begin{pmatrix} 2x \\ 2y \end{pmatrix}.$$

En consecuencia, la iteración del descenso del gradiente toma la forma

$$\begin{pmatrix} x_{k+1} \\ y_{k+1} \end{pmatrix} = \begin{pmatrix} x_k \\ y_k \end{pmatrix} - \alpha \begin{pmatrix} 2x_k \\ 2y_k \end{pmatrix},$$

lo cual equivale a

$$x_{k+1} = (1 - 2\alpha)x_k, \quad y_{k+1} = (1 - 2\alpha)y_k.$$

La convergencia hacia el origen ocurre siempre que

$$0 < \alpha < 1.$$

A pesar de su simplicidad y relevancia teórica, el método del descenso del gradiente posee ciertas limitaciones. En funciones mal condicionadas, la trayectoria de los iterados puede presentar oscilaciones pronunciadas y convergencia lenta, especialmente en regiones con geometría alargada o estrecha. Asimismo, en problemas no convexos el método puede converger hacia mínimos locales o puntos silla.

No obstante, el descenso del gradiente constituye la base conceptual de numerosos algoritmos modernos de optimización y aprendizaje automático, entre ellos el descenso estocástico del gradiente, los métodos con momentum, Nesterov, AdaGrad, RMSProp y Adam. Además, mantiene una estrecha relación con métodos de segundo orden como Newton y los algoritmos quasi-Newton.

Finalmente, el método admite una interpretación continua particularmente elegante. La iteración discreta puede verse como una aproximación numérica de la ecuación diferencial

$$\frac{dx(t)}{dt} = -\nabla f(x(t)),$$

conocida como flujo de gradiente (*gradient flow*). Esta ecuación describe una dinámica continua en la cual el sistema evoluciona siguiendo la dirección de máximo decrecimiento de la función energía f .

9.8.2. Regla de la cadena y retropropagación

El cálculo del gradiente en redes neuronales profundas se realiza mediante el algoritmo de retropropagación (*backpropagation*), el cual se basa en la aplicación sistemática de la regla de la cadena.

Sea una red expresada como:

$$f_{\theta} = f^{(L)} \circ \dots \circ f^{(1)}$$

el gradiente de la pérdida respecto a los parámetros de una capa l se obtiene propagando el error desde la salida hacia las capas anteriores.

Este procedimiento permite calcular eficientemente:

$$\nabla_{\theta^{(l)}} J(\theta)$$

para cada capa l de la red.

9.9. Regularización

En el entrenamiento de redes neuronales, es común que el modelo se ajuste excesivamente a los datos de entrenamiento, fenómeno conocido como *sobreajuste* (*overfitting*). En este caso, el modelo presenta un buen desempeño en los datos de entrenamiento, pero generaliza pobremente a datos no vistos.

La regularización consiste en un conjunto de técnicas diseñadas para mejorar la capacidad de generalización del modelo.

9.9.1. Sobreajuste

El sobreajuste ocurre cuando el modelo aprende no solo las características relevantes del problema, sino también el ruido presente en los datos. Esto suele suceder cuando el modelo tiene una alta capacidad de representación en relación con la cantidad de datos disponibles.

9.9.2. Penalización de parámetros (Weight Decay)

Una estrategia común consiste en añadir un término de penalización a la función de costo:

$$J(\theta) = \frac{1}{N} \sum_{i=1}^N \mathcal{L}(f_{\theta}(x_i), y_i) + \lambda \|\theta\|^2 \quad (9.20)$$

donde:

- $\lambda > 0$ es el parámetro de regularización
- $\|\theta\|^2$ es la norma cuadrática de los parámetros

Este término penaliza valores grandes de los parámetros, favoreciendo modelos más simples.

9.9.3. Dropout

El *dropout* es una técnica de regularización utilizada en redes neuronales profundas cuyo objetivo principal es reducir el sobreajuste y mejorar la capacidad de generalización del modelo. La idea esencial consiste en introducir aleatoriedad durante el entrenamiento mediante la desactivación temporal de neuronas. Como consecuencia, en cada iteración se entrena efectivamente una subred distinta de la arquitectura original.

Consideremos una capa neuronal cuya entrada es un vector

$$x \in \mathbb{R}^{H \times W \times C},$$

con matriz de pesos

$$W \in \mathbb{R}^{m \times d}$$

donde hay m neuronas. Y vector de sesgos

$$b \in \mathbb{R}^m.$$

La salida usual de la capa está dada por

$$h = \phi(Wx + b).$$

El mecanismo de dropout introduce un vector aleatorio

$$r = (r_1, \dots, r_m),$$

cuyas componentes son independientes y satisfacen

$$r_i \sim \text{Bernoulli}(p).$$

Aquí, el parámetro $p \in (0, 1]$ representa la probabilidad de conservar activa una neurona durante el entrenamiento. En consecuencia, cada neurona tiene probabilidad $1 - p$ de ser anulada temporalmente.

La nueva activación de la capa se define entonces por

$$\tilde{h} = r \odot h,$$

donde $\odot$ denota el producto componente a componente. De esta forma, si $r_i = 0$, la activación correspondiente desaparece completamente en esa iteración; si $r_i = 1$, la neurona permanece activa.

En implementaciones modernas suele utilizarse la variante denominada *inverted dropout*, en la cual las activaciones se reescalan durante el entrenamiento:

$$\tilde{h} = \frac{r}{p} \odot h.$$

La razón matemática de este reescalamiento es preservar el valor esperado de las activaciones. En efecto, puesto que

$$\mathbb{E}[r_i] = p,$$

se obtiene

$$\mathbb{E} \left[\frac{r_i}{p} h_i \right] = \frac{1}{p} \mathbb{E}[r_i] h_i = h_i.$$

Por tanto, el valor esperado de las activaciones permanece invariante:

$$\mathbb{E}[\tilde{h}] = h.$$

Gracias a esta propiedad, durante la etapa de inferencia no es necesario realizar ajustes adicionales: simplemente se utiliza la red completa sin eliminar neuronas.

Desde un punto de vista probabilístico, el dropout puede interpretarse como un procedimiento que entrena simultáneamente un gran conjunto de subredes. Si una red posee n neuronas susceptibles de ser desactivadas, entonces existen potencialmente

$$2^n$$

subredes distintas. El entrenamiento explícito de todas ellas sería computacionalmente inviable; sin embargo, el dropout proporciona una aproximación eficiente a un promedio implícito sobre dicho ensamble de modelos.

La relevancia de esta idea se encuentra en que las redes neuronales poseen una enorme capacidad de representación y, por ello, pueden memorizar patrones específicos del conjunto de entrenamiento. Cuando ciertas neuronas dependen excesivamente de otras, aparecen fenómenos de *co-adaptación*. El dropout rompe estas dependencias debido a que cualquier neurona puede desaparecer aleatoriamente durante el entrenamiento. Como consecuencia, cada unidad debe aprender representaciones más independientes y robustas.

Formalmente, el entrenamiento ya no minimiza una función de pérdida determinista, sino una pérdida esperada respecto de la aleatoriedad introducida por las máscaras de dropout. Si m representa una realización particular de la máscara aleatoria y θ denota el conjunto de parámetros de la red, entonces el problema de optimización puede escribirse como

$$\min_{\theta} \mathbb{E}_m [\mathcal{L}(f(x; m, \theta), y)].$$

Aquí, $f(x; m, \theta)$ representa la subred inducida por la máscara m . Así, cada actualización del descenso del gradiente se realiza sobre una arquitectura ligeramente distinta.

Si el descenso del gradiente estándar actualiza parámetros mediante

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}(\theta_t),$$

entonces bajo dropout la actualización toma la forma

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}(f(x; m_t, \theta_t), y),$$

donde m_t es una máscara aleatoria distinta en cada iteración. Este ruido adicional actúa como un mecanismo regularizador que dificulta el ajuste excesivamente preciso a los datos de entrenamiento.

En ciertos modelos simples, particularmente lineales, puede demostrarse que el dropout induce aproximadamente un efecto regularizador semejante a una penalización cuadrática sobre los pesos. En este sentido, su comportamiento se relaciona parcialmente con métodos de regularización tipo L^2 , aunque el mecanismo subyacente es distinto, ya que aquí la regularización surge a partir de una perturbación aleatoria multiplicativa.

Desde una perspectiva geométrica, el dropout favorece representaciones distribuidas y estables frente a perturbaciones. Una neurona no puede depender de la presencia permanente de otras unidades, de modo que la información relevante termina siendo compartida entre múltiples componentes de la red. Esto produce modelos menos frágiles y con mejor capacidad de generalización.

Aunque arquitecturas modernas incorporan otros mecanismos de regularización, como *Batch Normalization* y conexiones residuales, el dropout continúa siendo una herramienta fundamental, especialmente en redes densas tradicionales y en problemas donde la cantidad de datos disponibles es limitada.

9.10. Optimización

El proceso de entrenamiento de una red neuronal implica la resolución de un problema de optimización en alta dimensión. En la práctica, este problema se aborda mediante métodos iterativos basados en gradiente.

9.10.1. Descenso por gradiente estocástico (SGD)

En lugar de utilizar todo el conjunto de datos en cada iteración, el *Stochastic Gradient Descent* (SGD) emplea subconjuntos (mini-batches):

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}(f_{\theta}(x_i), y_i)$$

Este enfoque reduce el costo computacional y permite actualizaciones más frecuentes.

9.10.2. Mini-batch gradient descent

Una generalización del SGD consiste en utilizar lotes de tamaño intermedio:

$$\theta \leftarrow \theta - \eta \frac{1}{B} \sum_{i=1}^B \nabla_{\theta} \mathcal{L}(f_{\theta}(x_i), y_i) \quad (9.21)$$

donde B es el tamaño del lote.

9.10.3. Métodos adaptativos

Existen métodos de optimización que ajustan automáticamente la tasa de aprendizaje para cada parámetro. Uno de los más utilizados es *Adam*.

El algoritmo Adam combina ideas de momento y escalamiento adaptativo del gradiente. De manera simplificada, realiza actualizaciones de la forma:

$$\theta \leftarrow \theta - \eta \frac{m_t}{\sqrt{v_t} + \epsilon}$$

donde:

- m_t es el promedio exponencial de los gradientes
- v_t es el promedio exponencial de los gradientes al cuadrado
- ϵ es un término de estabilidad numérica

9.10.4. Tasa de aprendizaje

La tasa de aprendizaje η controla la magnitud de las actualizaciones. Su elección es crucial:

- valores pequeños producen convergencia lenta
- valores grandes pueden causar inestabilidad

En la práctica, se utilizan esquemas de decaimiento de la tasa de aprendizaje.

10 Caso de estudio

10.1. Construcción del conjunto de datos

El reconocimiento óptico de caracteres para sistemas de escritura indígenas, particularmente para el alfabeto Náhuatl, presenta una limitación inicial importante: no existe un conjunto de datos público, estandarizado y etiquetado que permita entrenar modelos de aprendizaje profundo. A diferencia de otros alfabetos, para los cuales existen repositorios ampliamente documentados, el alfabeto Náhuatl carece de recursos computacionales que puedan emplearse directamente en tareas de clasificación automática.

Ante esta situación, fue necesario construir un conjunto de datos propio. La decisión de generar un dataset sintético se tomó por dos razones principales:

1. Proporcionar un número suficiente de ejemplos para el entrenamiento de la red neuronal.
2. Introducir variaciones geométricas que permitan al modelo generalizar ante deformaciones presentes en documentos reales.

10.1.1. Generación de las muestras

Inicialmente se diseñaron de forma manual 25 ejemplos base para cada carácter del alfabeto. Posteriormente, a cada muestra se le aplicaron transformaciones geométricas y perturbaciones aleatorias para incrementar la variabilidad del conjunto de datos.

La Figura @fig:dataset muestra algunos ejemplos de las imágenes generadas.

10.2. Carga y organización de los datos

Una vez construido el conjunto de datos, las imágenes se almacenaron siguiendo una estructura jerárquica de directorios, donde cada carpeta representa una clase distinta del alfabeto.

El cargado de las imágenes se realizó mediante la función `image_dataset_from_directory` de TensorFlow:

```
train_ds = tf.keras.utils.image_dataset_from_directory(
    data_dir,
    validation_split=0.2,
    subset="training",
    seed=123,
```

```

        image_size=(28,28),
        batch_size=32,
        color_mode="grayscale"
    )

val_ds = tf.keras.utils.image_dataset_from_directory(
    data_dir,
    validation_split=0.2,
    subset="validation",
    seed=123,
    image_size=(28,28),
    batch_size=32,
    color_mode="grayscale"
)

```

La elección de una partición de entrenamiento-validación de 80 %-20 % constituye una práctica común en aprendizaje automático, ya que permite disponer de un número suficiente de ejemplos para el ajuste de los parámetros del modelo y, al mismo tiempo, reservar un conjunto independiente para evaluar la capacidad de generalización.

Asimismo, las imágenes se convirtieron a escala de grises. Dado que los glifos del alfabeto Náhuatl se distinguen principalmente por su geometría y no por información cromática, el uso de un solo canal reduce el costo computacional sin sacrificar información relevante.

10.3. Normalización y aumentación de datos

Antes de alimentar las imágenes a la red neuronal, se realizó un proceso de normalización:

```
normalization_layer = tf.keras.layers.Rescaling(1./255)
```

Esta transformación reescala los valores de intensidad al intervalo $[0, 1]$, lo cual mejora la estabilidad numérica del proceso de entrenamiento y facilita la convergencia del algoritmo de optimización.

Adicionalmente, se implementó un mecanismo de aumentación de datos en línea:

```

data_augmentation = tf.keras.Sequential([
    tf.keras.layers.RandomRotation(0.05),
    tf.keras.layers.RandomZoom(0.1),
    tf.keras.layers.RandomTranslation(0.05, 0.05)
])

```

La introducción de pequeñas rotaciones, cambios de escala y traslaciones tiene como objetivo simular las variaciones presentes en documentos escaneados o en la escritura manual. De

esta manera, el modelo aprende representaciones más robustas y reduce su tendencia al sobreajuste.

Finalmente, los datos fueron almacenados temporalmente en memoria y preparados para alimentar la GPU:

```
train_ds = train_ds.cache().shuffle(5000).prefetch(AUTOTUNE)
val_ds = val_ds.cache().prefetch(AUTOTUNE)
```

El uso de `cache()` evita que las imágenes sean leídas repetidamente desde el disco, mientras que `prefetch()` permite superponer las operaciones de lectura de datos con el entrenamiento de la red, disminuyendo el tiempo total de ejecución.

11 Arquitectura de la red neuronal convolucional

La arquitectura propuesta recibe imágenes de tamaño $28 \times 28 \times 1$ y está compuesta por tres bloques convolucionales seguidos de un clasificador denso.

```
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(28,28,1)),

    tf.keras.layers.Conv2D(32,3,activation='relu'),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.SpatialDropout2D(0.25),

    tf.keras.layers.Conv2D(64,3,activation='relu'),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.SpatialDropout2D(0.25),

    tf.keras.layers.Conv2D(128,3,activation='relu'),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.SpatialDropout2D(0.25),

    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(256, activation='relu'),
    tf.keras.layers.Dropout(0.5),
    tf.keras.layers.Dense(num_classes, activation='softmax')
])
```

11.1. Primer bloque convolucional

La primera capa convolucional emplea 32 filtros de tamaño 3×3 :

```
tf.keras.layers.Conv2D(32,3,activation='relu')
```

Este bloque se encarga de detectar características de bajo nivel, tales como bordes, esquinas y pequeños segmentos de los trazos.

La incorporación de una capa de normalización por lotes:

```
tf.keras.layers.BatchNormalization()
```

permite estabilizar la distribución de las activaciones internas, acelerando el proceso de convergencia.

Posteriormente, la operación de *max pooling*

```
tf.keras.layers.MaxPooling2D()
```

reduce la dimensionalidad espacial de las representaciones, disminuyendo el número de parámetros y haciendo al modelo menos sensible a pequeñas traslaciones.

Finalmente, se incorpora un mecanismo de regularización:

```
tf.keras.layers.SpatialDropout2D(0.25)
```

que desactiva mapas de características completos de manera aleatoria, obligando a la red a aprender representaciones más robustas.

11.2. Segundo y tercer bloque convolucional

Los bloques posteriores incrementan progresivamente el número de filtros hasta 64 y 128, respectivamente. El aumento en la profundidad de la red permite capturar patrones cada vez más complejos, tales como combinaciones de curvas, intersecciones y configuraciones geométricas características de cada glifo.

11.3. Clasificador

Las características extraídas se transforman en un vector unidimensional:

```
tf.keras.layers.Flatten()
```

y posteriormente se procesan mediante una capa densa:

```
tf.keras.layers.Dense(256, activation='relu')
```

La capa de salida utiliza la función Softmax:

```
tf.keras.layers.Dense(num_classes, activation='softmax')
```

la cual produce una distribución de probabilidad sobre todas las clases del alfabeto.

12 Entrenamiento

El modelo se compiló utilizando el optimizador Adam:

```
model.compile(  
    optimizer='adam',  
    loss='sparse_categorical_crossentropy',  
    metrics=['accuracy']  
)
```

La función de pérdida de entropía cruzada dispersa se seleccionó debido a que las etiquetas de las imágenes se encuentran codificadas como enteros y no mediante vectores *one-hot*.

Para evitar el sobreajuste se implementó un mecanismo de detención temprana:

```
early_stop = EarlyStopping(  
    monitor='val_loss',  
    patience=7,  
    restore_best_weights=True  
)
```

La elección de siete épocas de paciencia permite al modelo continuar el aprendizaje ante pequeñas oscilaciones de la pérdida de validación, evitando finalizar el entrenamiento de forma prematura.

Asimismo, el mejor modelo obtenido se almacenó automáticamente:

```
checkpoint = ModelCheckpoint(  
    ruta_modelo,  
    monitor='val_loss',  
    save_best_only=True  
)
```

13 Sistema de inferencia interactivo

Finalmente, se desarrolló una interfaz interactiva en Google Colab que permite dibujar caracteres del alfabeto Náhuatl y obtener predicciones en tiempo real.

La imagen generada por el usuario se redimensiona a 28×28 píxeles:

```
img_resized = cv2.resize(img, (28,28))
img_normalized = img_resized / 255.0
img_input = np.expand_dims(img_normalized, axis=(0,-1))
```

Posteriormente, el modelo produce un vector de probabilidades:

```
predicciones = model.predict(img_input)
```

y se determina la clase más probable mediante:

```
idx_ganador = np.argmax(predicciones)
```

Además de la predicción principal, la interfaz presenta las tres clases con mayor probabilidad, proporcionando información adicional sobre la confianza y las posibles ambigüedades del modelo.

14 Conclusiones

15 Referencias